

ACTIVIDAD FINAL – MÓDULO DTPR

DataPath Tipo R — Procesador MIPS Simple

Materia: Arquitectura de Computadoras

Actividad: Actividad Final del Módulo DTPR

Herramienta: ModelSim – Intel FPGA Starter Edition 2020.1

Lenguaje: Verilog HDL

Maestro: Jorge Ernesto López Arce Delgado

Integrantes:

Diego Israel González Sánchez

Derek Gabriel Casillas López

Iván Samuel Ortiz Orellana

Omar Luna Reyes

24/04/2026

1. Introducción

El diseño de un procesador moderno comienza con la comprensión de sus componentes fundamentales: la Unidad Aritmético-Lógica (ALU), la Memoria de Datos y el Banco de Registros. Una vez que se dominan estos elementos por separado, el siguiente paso es integrarlos en una estructura denominada DataPath (camino de datos), que define cómo fluye la información entre ellos durante la ejecución de una instrucción.

El presente reporte documenta la construcción del Single DataPath Tipo R (DPTR), que representa una versión simplificada del camino de datos del procesador MIPS capaz de decodificar y ejecutar instrucciones del tipo R. Este diseño sigue la arquitectura de ciclo único descrita en textos clásicos de arquitectura de computadoras como Patterson & Hennessy.

Las instrucciones implementadas en este DataPath son: ADD (suma entera), SUB (resta entera), AND (conjunción lógica), OR (disyunción lógica) y SLT (Set on Less Than). Estas cinco instrucciones cubren las operaciones aritméticas y lógicas más básicas del ISA MIPS y son suficientes para demostrar el principio de funcionamiento del DataPath.

El proceso de construcción del DPTR requirió la integración de módulos ya existentes (ALU, BancoReg y Mem) con tres módulos nuevos: la Unidad de Control, la ALU Control y un Multiplexor 2:1 de 32 bits. Cada módulo fue desarrollado y probado de forma independiente antes de integrarse en el sistema completo.

2. Descripción de las Instrucciones Tipo R

2.1 Formato General de la Instrucción

Las instrucciones tipo R en la arquitectura MIPS tienen un ancho fijo de 32 bits. Estos 32 bits no están divididos físicamente; es el hardware del procesador (en este caso el módulo DPTR) quien extrae cada campo mediante selección de bits. Los seis campos son:

Tabla 1. Formato de Instrucción Tipo R – campos y tamaños en bits.

OpCode [31:26]	RS [25:21]	RT [20:16]	RD [15:11]	Shamt [10:6]	Funct [5:0]
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

La descripción de cada campo es:

- **OpCode:** Siempre vale 000000 para instrucciones tipo R. Indica a la Unidad de Control que la operación específica está definida por el campo Funct.
- **RS:** Especifica el número del primer registro fuente (operando 1).
- **RT:** Especifica el número del segundo registro fuente (operando 2).
- **RD:** Especifica el número del registro destino donde se almacenará el resultado.
- **Shamt:** Campo de desplazamiento (shift amount). No se usa en las instrucciones implementadas; siempre es 00000.
- **Funct:** Código de función. Combinado con la señal ALUOp proveniente de la Unidad de Control, determina la operación que realiza la ALU.

[Imagen: Diagrama del Formato de Instrucción Tipo R (32 bits)]

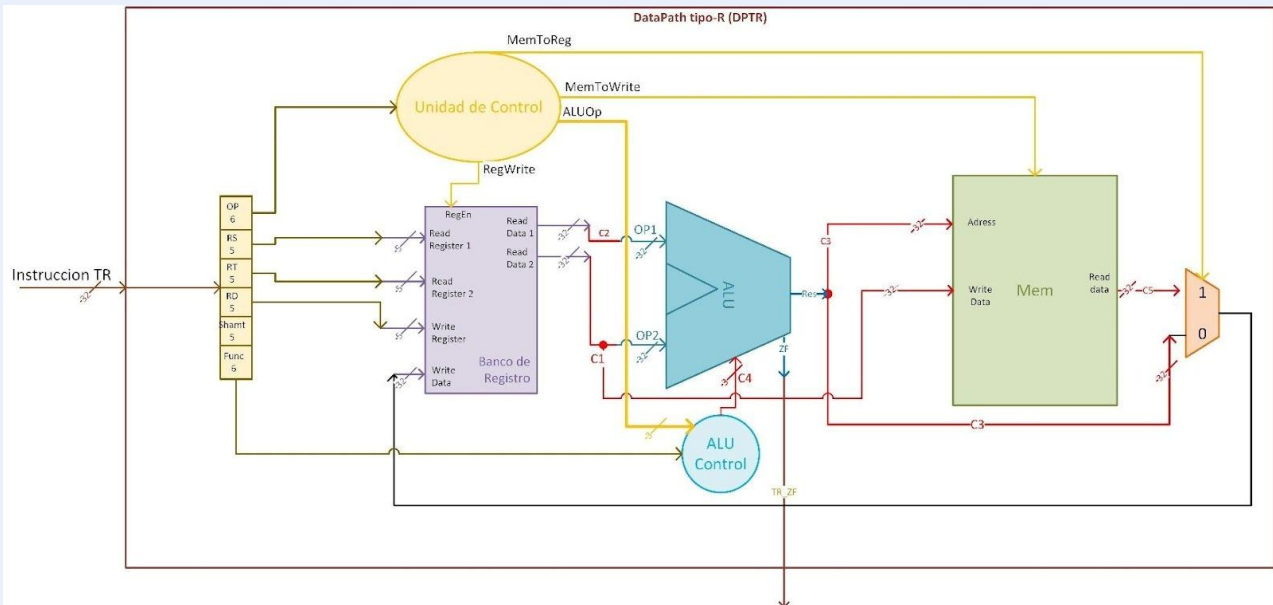


Figura 1. Diagrama del formato de instrucción tipo R mostrando la distribución de los 32 bits en sus seis campos.

2.2 Instrucciones Implementadas

Las cinco instrucciones implementadas en este DataPath son:

- ADD — Suma entera de dos registros. Resultado = RS + RT.
- SUB — Resta entera. Resultado = RS - RT.
- AND — Conjunción lógica bit a bit. Resultado = RS & RT.
- OR — Disyunción lógica bit a bit. Resultado = RS | RT.
- SLT — Set on Less Than. Coloca 1 en RD si RS < RT, de lo contrario coloca 0.

2.3 Instrucción SLT — Set on Less Than

La instrucción SLT (Set on Less Than) es una instrucción de comparación. Su propósito es comparar dos registros y escribir el resultado de esa comparación (0 o 1) en un tercer registro. Es fundamental para implementar estructuras de control condicionales (if, while, for) cuando se compila código de alto nivel a ensamblador MIPS.

En el ensamblador MIPS, su sintaxis es: `slt $RD, $RS, $RT`

Esta instrucción es fundamental para la implementación de estructuras de control condicionales como if-else y bucles. Antes de la existencia de instrucciones de comparación dedicadas, se requerían múltiples instrucciones para lograr el mismo efecto.

Su código de función (Funct) es 101010 en binario (0x2A en hexadecimal). El OpCode permanece en 000000 al ser una instrucción tipo R.

Ejemplo de uso: si R24 = 24 y R25 = 25, entonces `slt $12, $24, $25` almacenará 1 en el registro \$12 porque 24 es efectivamente menor que 25.

3. El Operador Ternario en Verilog

3.1 Definición

El operador ternario es una estructura de control compacta que permite expresar una condición y sus dos posibles resultados en una sola línea. Es equivalente funcional a una sentencia if-else pero con una sintaxis más concisa y directa.

Su sintaxis general es:

```
resultado = condicion ? valor_si_verdadero : valor_si_falso;
```

En Verilog, este operador se evalúa de forma combinacional dentro de sentencias assign o bloques always @(*). El sintetizador lo convierte en hardware equivalente a un multiplexor 2:1.

3.2 Aplicación en la ALU — Instrucción SLT

Para implementar la instrucción SLT en la ALU sin recurrir a una sentencia if-else extensa, se utiliza el operador ternario de la siguiente manera:

```
3'b100: Res = (OP1 < OP2) ? 32'd1 : 32'd0; // SLT
```

Esta línea indica que cuando el control de la ALU recibe el código 100 (binario), debe evaluar si OP1 es menor que OP2. Si la condición es verdadera, el resultado es 1 extendido a 32 bits; de lo contrario, es 0. La comparación < en Verilog trata los operandos como enteros sin signo por defecto, lo cual es consistente con el comportamiento de SLT en MIPS.

3.3 Ventajas del Operador Ternario en Hardware

El uso del operador ternario en Verilog es equivalente funcionalmente a un multiplexor de 2 entradas. El sintetizador genera circuito combinacional eficiente, sin necesidad de flip-flops ni lógica secuencial. Además, mejora la legibilidad del código al mantener la lógica de selección en una sola expresión.

4. Proceso de Compilación

4.1 Compilación en Lenguajes de Alto Nivel

La compilación es el proceso mediante el cual un programa escrito en un lenguaje de alto nivel (como C o C++) es transformado automáticamente en instrucciones de lenguaje máquina que el procesador puede ejecutar directamente. Este proceso se divide en las siguientes etapas:

- **Análisis Léxico:** El código fuente se descompone en unidades mínimas llamadas tokens (palabras clave, identificadores, operadores, literales).
- **Análisis Sintáctico:** Los tokens se organizan en una estructura jerárquica (árbol sintáctico abstracto, AST) que verifica que el código sigue la gramática del lenguaje.
- **Análisis Semántico:** Se verifica que el programa tenga sentido lógico: tipos compatibles, variables declaradas, ámbitos correctos.
- **Generación de Código Intermedio:** Se produce una representación intermedia independiente de la arquitectura destino.
- **Optimización:** Se mejora el código intermedio para reducir el número de instrucciones, el uso de registros y el consumo de memoria.
- **Generación de Código Máquina:** Se produce el código en lenguaje ensamblador o binario específico para la arquitectura destino (en este caso MIPS).

4.2 Relación con el DataPath Tipo R

El DataPath implementado en esta actividad representa la etapa final del proceso de compilación: el hardware que ejecuta las instrucciones binarias generadas por el compilador. Cada instrucción de 32 bits que llega al DPTR es exactamente el código máquina que un compilador de C para MIPS produciría a partir de operaciones aritméticas simples.

Esto demuestra la importancia de comprender el hardware: el compilador debe conocer el conjunto de instrucciones (ISA) del procesador para generar código correcto y eficiente.

5. Descripción de los Módulos del DataPath

5.1 Arquitectura General del DPTR

El DataPath Tipo R integra seis módulos que trabajan en conjunto para decodificar y ejecutar una instrucción tipo R completa. El flujo general es: la instrucción de 32 bits entra al DPTR, el OpCode se envía a la Unidad de Control, el Funct a la ALU Control, y los campos RS, RT, RD al Banco de Registros. La ALU opera sobre los datos leídos del banco y el resultado regresa al banco a través del multiplexor.

Tabla 2. Resumen de módulos del DPTR con sus entradas y salidas.

Módulo	Entradas	Salidas
UnidadDeControl	OpCode [5:0]	MemToReg [1], RegWrite [1], MemToWrite [1], ALUOp [2:0]
ALUControl	Funct [5:0], ALUOp [2:0]	ALUCtrl [2:0]
Mux2_1_32	In0 [31:0], In1 [31:0], Sel [1]	Out [31:0]
BancoReg	ReadReg1 [4:0], ReadReg2 [4:0], WriteReg [4:0], WriteData [31:0], RegWrite [1]	ReadData1 [31:0], ReadData2 [31:0]
ALU	OP1 [31:0], OP2 [31:0], ALUCtrl [2:0]	Res [31:0], ZF [1]
Mem	Address [31:0], WriteData [31:0], MemWrite [1]	ReadData [31:0]
DPTR	InstruccionTR [31:0]	(Integra todos los módulos anteriores)

Figura 2. Diagrama del DataPath Tipo R mostrando la interconexión entre la Unidad de Control, ALU Control, Banco de Registros, ALU, Memoria y Multiplexor.

5.2 Unidad de Control (UnidadDeControl.v)

La Unidad de Control recibe los 6 bits del campo OpCode de la instrucción (bits 31:26). Para instrucciones tipo R (OpCode = 000000), activa RegWrite = 1 para permitir la escritura en el Banco de Registros, mantiene MemToReg = 0 para que el dato de escritura provenga de la ALU y no de la memoria, y mantiene MemToWrite = 0 para no escribir en memoria. Además envía ALUOp = 010 a la ALU Control para indicar que debe decodificar el campo Funct.

Tabla 3. Señales de control generadas por la UC para instrucciones tipo R.

Señal	Valor (tipo R)	Módulo destino	Efecto	Descripción
RegWrite	1	Banco de Registros	Activo	Habilita escritura en el registro destino RD
MemToReg	0	Mux Final	Sel=0	Selecciona resultado de ALU (no de memoria)
MemToWrite	0	Memoria	Inactivo	No escribe en memoria
ALUOp	010	ALU Control	Tipo R	Indica instrucción tipo R; ALUCtrl depende de Funct

5.3 ALU Control (ALUControl.v)

La ALU Control recibe dos entradas: los 6 bits del campo Funct (bits 5:0 de la instrucción) y la señal ALUOp de 3 bits proveniente de la Unidad de Control. Cuando ALUOp = 010 (instrucción tipo R), la ALU Control decodifica el Funct para generar la señal ALUCtrl de 3 bits que determina la operación de la ALU.

Tabla 4. Tabla de decodificación de la ALU Control para instrucciones tipo R.

Instrucción	Funct (binario)	Funct (hex)	ALUCtrl	Operación ALU
ADD	100000	0x20	000	Suma
SUB	100010	0x22	001	Resta
AND	100100	0x24	010	AND bit a bit
OR	100101	0x25	011	OR bit a bit
SLT	101010	0x2A	100	Comparación menor que

5.4 Multiplexor 2:1 de 32 bits (Mux2_1_32.v)

Este módulo recibe dos valores de 32 bits (el resultado de la ALU en In0 y el dato leído de la memoria en In1) y selecciona cuál de los dos se envía de vuelta al Banco de Registros como dato a escribir. En instrucciones tipo R, MemToReg = 0 siempre, por lo que siempre se selecciona el resultado de la ALU. Su implementación en Verilog usa el operador ternario:

```
assign Out = (Sel) ? In1 : In0;
```

5.5 Banco de Registros (BancoReg.v)

El Banco de Registros contiene 32 registros de 32 bits cada uno. En la simulación, cada registro se inicializa con su propio índice ($R[i] = i$), lo que facilita la verificación de los resultados: $R1=1$, $R2=2$, ..., $R31=31$. Tiene dos puertos de lectura asíncronos y un puerto de escritura sincronizado con la señal RegWrite. El registro R0 está protegido y no puede ser modificado.

5.6 Memoria de Datos (Mem.v)

La memoria de datos no es utilizada activamente en las instrucciones tipo R (MemWrite = 0 siempre). Sin embargo, se incluye en el DataPath para completar la arquitectura y está inicializada en cero para evitar valores indefinidos ('X') en la simulación. La lectura es asíncrona y la escritura está controlada por MemWrite.

6. Tablas de Instrucciones del TestBench

6.1 Instrucciones en Formato Ensamblador

La siguiente tabla muestra las 10 instrucciones enviadas al DPTR durante la simulación, expresadas en formato ensamblador MIPS estándar. Los valores entre paréntesis indican el valor inicial del registro en el Banco de Registros ($R[i] = i$).

Tabla 5. Instrucciones del TestBench en formato ensamblador MIPS.

#	Instrucción Ensamblador	Operación	RS – fuente 1	RT – fuente 2	RD – destino
1	add \$8, \$16, \$17	ADD	\$16 ($R16 = 16$)	\$17 ($R17 = 17$)	\$8
2	sub \$9, \$18, \$19	SUB	\$18 ($R18 = 18$)	\$19 ($R19 = 19$)	\$9
3	and \$10, \$20, \$21	AND	\$20 ($R20 = 20$)	\$21 ($R21 = 21$)	\$10
4	or \$11, \$22, \$23	OR	\$22 ($R22 = 22$)	\$23 ($R23 = 23$)	\$11
5	slt \$12, \$24, \$25	SLT	\$24 ($R24 = 24$)	\$25 ($R25 = 25$)	\$12
6	add \$8, \$17, \$18	ADD	\$17 ($R17 = 17$)	\$18 ($R18 = 18$)	\$8
7	sub \$9, \$20, \$21	SUB	\$20 ($R20 = 20$)	\$21 ($R21 = 21$)	\$9
8	and \$10, \$20, \$21	AND	\$20 ($R20 = 20$)	\$21 ($R21 = 21$)	\$10
9	or \$11, \$22, \$23	OR	\$22 ($R22 = 22$)	\$23 ($R23 = 23$)	\$11
10	slt \$12, \$24, \$25	SLT	\$24 ($R24 = 24$)	\$25 ($R25 = 25$)	\$12

6.2 Instrucciones en Código Máquina

La siguiente tabla muestra las mismas instrucciones codificadas en formato binario de 32 bits, desglosadas campo por campo según el formato de instrucción tipo R, junto con su representación hexadecimal completa que es la entrada real del TestBench.

Tabla 6. Instrucciones del TestBench en código máquina (binario por campos y hexadecimal).

#	OpCode (6)	RS (5)	RT (5)	RD (5)	Shamt (5)	Funct (6)	Hex completo
1	000000	10000	10001	01000	00000	100000	0x02114020
2	000000	10010	10011	01001	00000	100010	0x02534822
3	000000	10100	10101	01010	00000	100100	0x02955024
4	000000	10110	10111	01011	00000	100101	0x02D75825
5	000000	11000	11001	01100	00000	101010	0x0319602A
6	000000	10001	10010	01000	00000	100000	0x02324020
7	000000	10100	10101	01001	00000	100010	0x02544822
8	000000	10100	10101	01010	00000	100100	0x02955024
9	000000	10110	10111	01011	00000	100101	0x02D75825
10	000000	11000	11001	01100	00000	101010	0x0319602A

6.3 Resultados Esperados de la Simulación

Con base en la inicialización del Banco de Registros ($R[i] = i$), los resultados esperados para cada instrucción son:

Tabla 7. Resultados esperados de la simulación en formato decimal.

#	Instrucción	RD1 (dec)	RD2 (dec)	ALURes (dec)	Descripción
1	ADD \$8 = \$16 + \$17	16	17	33	$16 + 17 = 33$
2	SUB \$9 = \$18 - \$19	18	19	-1	$18 - 19 = -1$
3	AND \$10 = \$20 & \$21	20	21	20	$10100 \text{ AND } 10101 = 10100 = 20$
4	OR \$11 = \$22 \$23	22	23	23	$10110 \text{ OR } 10111 = 10111 = 23$
5	SLT \$12 = \$24 < \$25	24	25	1	$24 < 25 \rightarrow \text{resultado } 1$
6	ADD \$8 = \$17 + \$18	17	18	35	$17 + 18 = 35$
7	SUB \$9 = \$20 - \$21	20	21	-1	$20 - 21 = -1$
8	AND \$10 = \$20 & \$21	20	21	20	$10100 \text{ AND } 10101 = 10100 = 20$
9	OR \$11 = \$22 \$23	22	23	23	$10110 \text{ OR } 10111 = 10111 = 23$
10	SLT \$12 = \$24 < \$25	24	25	1	$24 < 25 \rightarrow \text{resultado } 1$

7. Análisis de Resultados de la Simulación

La simulación en ModelSim confirmó el correcto funcionamiento del DataPath Tipo R. A continuación se analizan las observaciones más relevantes de la ventana Wave:

- Las señales de control se comportan de forma consistente: $\text{RegWrite} = 1$, $\text{MemToReg} = 0$ y $\text{MemToWrite} = 0$ permanecen constantes durante toda la simulación, lo cual es correcto para instrucciones tipo R.
- La señal $\text{ALUOp} = 010$ (2 en decimal) se mantiene constante, delegando la decisión de la operación a la ALU Control mediante el campo Funct.
- La primera instrucción (ADD \$8 = \$16 + \$17) produjo $\text{RD1} = 16$, $\text{RD2} = 17$ y $\text{ALURes} = 33$, resultado correcto visible en la simulación.
- La instrucción SLT produjo el resultado esperado de 1 cuando el primer operando es menor que el segundo, verificando la correcta implementación del operador ternario.
- El Banco de Registros (Memory.../registros) mostró la inicialización correcta $R[i] = i$, verificable en la segunda captura de ModelSim.
- El Multiplexor seleccionó correctamente el resultado de la ALU ($\text{WData} = \text{ALURes}$) en todos los casos, ya que $\text{MemToReg} = 0$ siempre.

[Imagen: Captura de la ventana Wave de ModelSim – señales en formato decimal]

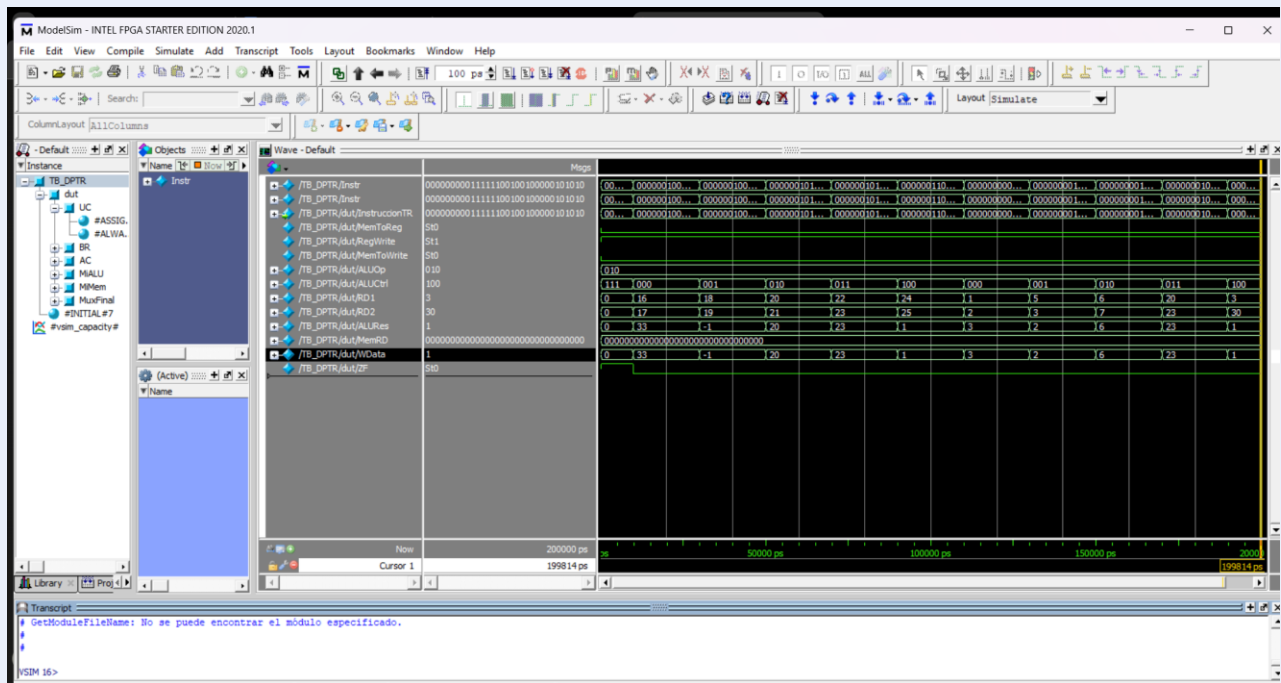


Figura 3. Ventana Wave de ModelSim mostrando las señales de control y datos del DPTR para las 10 instrucciones.

[Imagen: Captura del Memory Data de ModelSim – Banco de Registros inicializado $R[i]=i$]

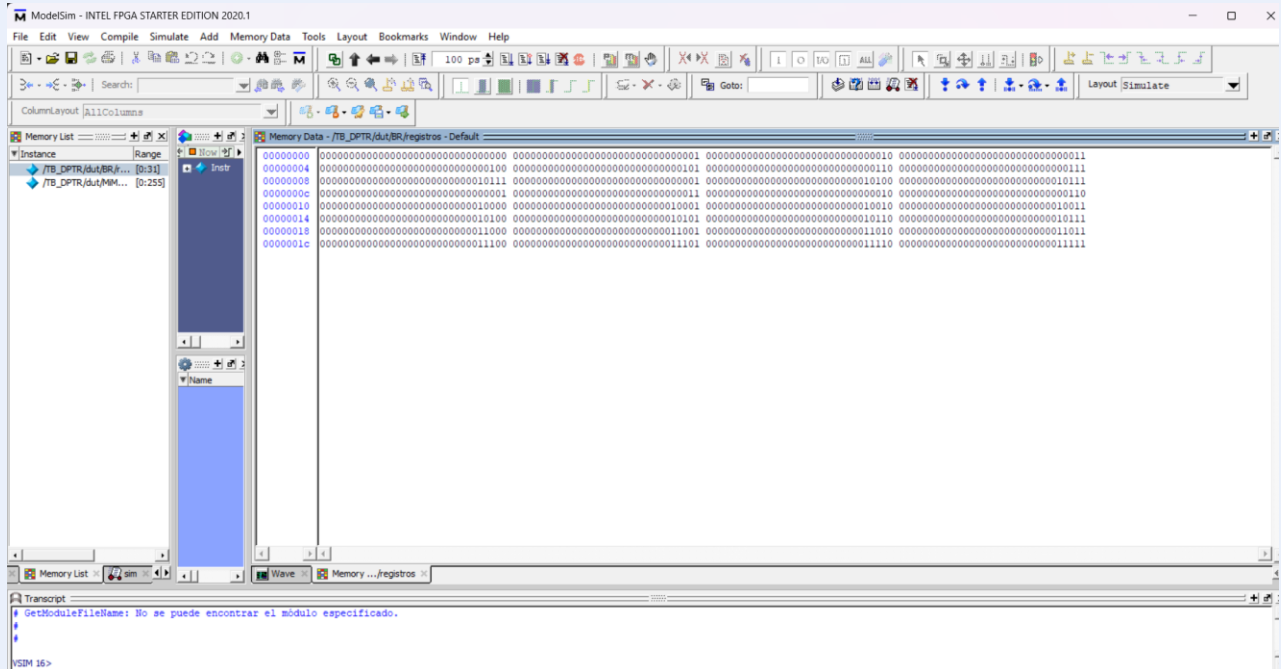


Figura 4. Vista de Memory Data en ModelSim confirmando la inicialización del Banco de Registros con $R[i] = i$ ($R0=0$, $R1=1$, $R2=2$, ... $R31=31$).

8. Conclusiones

La implementación del DataPath Tipo R en Verilog permitió comprender de manera práctica cómo los diferentes componentes de un procesador trabajan en conjunto para ejecutar una instrucción. El hecho de ver cómo una cadena de 32 bits se descompone en campos, activa señales de control y produce un resultado verificable en registros hace muy concreto algo que en teoría puede parecer abstracto.

La instrucción SLT evidenció la importancia del operador ternario como herramienta de síntesis: permite describir lógica de multiplexor de forma directa y legible, y el sintetizador genera hardware eficiente sin ambigüedades. Modificar la ALU para agregar esta instrucción fue más sencillo de lo esperado gracias a la estructura del case en Verilog.

Este DataPath sienta las bases para versiones más complejas del procesador MIPS, donde se agregarán instrucciones tipo I (carga, almacenamiento, saltos condicionales) y eventualmente un contador de programa (PC) para convertirlo en un procesador de ciclo único completo.

En conjunto, esta actividad integra los conceptos de diseño digital, arquitectura de conjunto de instrucciones, síntesis de hardware y verificación por simulación, demostrando que el conocimiento del ISA y del hardware son dos caras inseparables de la misma moneda en el diseño de sistemas digitales.

Referencias

- Patterson, D. A., & Hennessy, J. L. (2014). Computer Organization and Design: The Hardware/Software Interface (5th ed.). Morgan Kaufmann.
- Harris, D. M., & Harris, S. L. (2012). Digital Design and Computer Architecture (2nd ed.). Morgan Kaufmann.
- Ciletti, M. D. (2011). Advanced Digital Design with the Verilog HDL (2nd ed.). Pearson Education.
- Intel Corporation. (2020). ModelSim PE Student Edition User Manual. Recuperado de <https://www.intel.com/>
- Mano, M. M., & Ciletti, M. D. (2013). Digital Design: With an Introduction to the Verilog HDL (5th ed.). Pearson.